

面向 VUL-RAG 漏洞检测的倒数排名加权综合决策方法

摘要 基于大语言模型 (LLM) 的漏洞检测方法近年来取得显著进展, 但如何有效利用检索增强生成 (RAG) 框架中的历史漏洞知识, 仍面临决策鲁棒性不足等挑战. VUL-RAG 作为知识级 RAG 漏洞检测的代表性方法, 通过抽取功能语义、漏洞成因与修复方案等多维知识, 显著提升了 LLM 区分漏洞代码与相似修复代码的能力. 然而, 在其开源实现中, 检测决策层采用早退 (early return) 与“一票否决”策略, 使得任一检索知识项均可单方面决定最终结果, 当检索到不相关或描述模糊的知识时易产生误报, 制约 Pair Accuracy 的进一步提升. 针对上述问题, 本文提出一种面向 VUL-RAG 的倒数排名加权综合决策方法 (WRAD, Weighted Reciprocal-rank Aggregated Decision). 该方法保留 VUL-RAG 原有的知识抽取、BM25 三路检索与 LLM 双通道推理 (漏洞成因检测与修复措施检测) 流程, 仅替换决策层逻辑: 对检索排序后的全部知识项赋予倒数排名权重, 分别累加漏洞证据与修复证据的加权得分, 通过比较二者强度做出最终判定. 该方法无需额外训练或超参数调优, 对上下游模块完全兼容. 在 CWE-401 (内存泄漏) 场景下, 本文基于 Ollama 本地部署模型复现 VUL-RAG 流水线, 并在原版知识库与 DeepSeek-Coder-V2 检测配置下进行对比实验. 结果表明, 相比原始早退决策策略, WRAD 将 Pair Accuracy 从 0.3077 提升至 0.3462 (相对提升 12.5%), 真阳性 (TP) 由 13 增至 20, 漏报 (FN) 由 13 降至 6, F1 由 0.6186 提升至 0.6581. 进一步在 CWE-264 的 31 对样本上采用原版知识库、DeepSeek-Coder-V2 16k 与 WRAD 进行补充实验, 获得 0.3226 的 Pair Accuracy, 表明该决策方法能够迁移至另一类漏洞数据; 但由于缺少同配置的原始早退基线, 该结果仅作为跨 CWE 适用性的初步证据. 实验还系统分析了不同知识来源与检测模型组合对复现效果的影响, 为本地轻量化部署场景下的 RAG 漏洞检测优化提供了可行路径.

关键词 漏洞检测; 检索增强生成; VUL-RAG; 加权决策; CWE-401; CWE-264; 大语言模型

中图法分类号 TP311

Weighted Reciprocal-rank Aggregated Decision for VUL-RAG Vulnerability Detection

Abstract Large language model (LLM)-based vulnerability detection has made significant progress, yet effectively leveraging retrieved vulnerability knowledge in retrieval-augmented generation (RAG) frameworks remains challenging due to insufficient decision robustness. VUL-RAG, a representative knowledge-level RAG approach, distills multi-dimensional knowledge including functional semantics, vulnerability root causes, and fixing solutions, substantially improving LLMs' ability to distinguish vulnerable code from similar patched code. However, its open-source implementation adopts an early-return strategy with a “single-vote veto” mechanism in the decision layer, allowing any retrieved knowledge item to unilaterally determine the final result. When irrelevant or imprecise knowledge is retrieved, this design tends to produce false positives and limits further improvement in Pair Accuracy. To address this issue, we propose a Weighted Reciprocal-rank Aggregated Decision (WRAD) method for VUL-RAG. WRAD preserves the original knowledge extraction, BM25 triple-path retrieval, and LLM dual-channel reasoning pipeline, replacing only the decision layer: all retrieved knowledge items are assigned reciprocal-rank weights, and weighted scores for vulnerability evidence and fixing evidence are accumulated and compared to produce the final verdict. The method requires no additional training or hyperparameter tuning and remains fully compatible with upstream and downstream modules. Under the CWE-401 (memory leak) scenario, we reproduce the VUL-RAG pipeline using locally deployed Ollama models and conduct comparative experiments with the original knowledge base and DeepSeek-Coder-V2 as the detection model. Results show that compared to the original early-return strategy, WRAD improves Pair Accuracy from 0.3077 to 0.3462 (a relative gain of 12.5%), increases true positives from 13 to 20, reduces false negatives

from 13 to 6, and improves F1 from 0.6186 to 0.6581. A supplementary experiment on 31 CWE-264 pairs, using the original knowledge base, DeepSeek-Coder-V2 with a 16k context window, and WRAD, achieves a Pair Accuracy of 0.3226. This provides preliminary evidence that the decision method can be applied to another vulnerability category; however, without an early-return baseline under the same configuration, it should not be interpreted as proof of cross-CWE performance improvement. We also systematically analyze the impact of different knowledge sources and detection model combinations on reproduction performance, providing a practical path for optimizing RAG-based vulnerability detection under lightweight local deployment.

Key words vulnerability detection; retrieval-augmented generation; VUL-RAG; weighted decision; CWE-401; CWE-264; large language model

1 引言

软件漏洞是威胁信息系统安全的核心因素之一。随着开源软件规模的持续扩大,人工审计已难以满足漏洞发现的需求,自动化漏洞检测因此成为软件工程与安全领域的研究热点 [1, 2, 4]。传统静态分析工具在大规模工程应用中仍需面对误报、漏报以及复杂告警难以理解等问题 [3]。近年来, GPT-4、Qwen2.5-Coder 等通用与代码大模型展现出较强的代码理解和推理能力 [5, 6], 并逐渐被用于漏洞检测研究 [7, 11]。然而, Du 等人 [7] 的实证研究表明, 即便是最先进的 LLM, 在区分“漏洞代码”与“结构相似但已修复的代码”时, Pair Accuracy 仍低至 0.06–0.14, 接近随机猜测水平, 表明 LLM 难以独立捕获漏洞的根因语义。

为突破上述瓶颈, Du 等人 [7] 提出 VUL-RAG (Vulnerability Retrieval-Augmented Generation) 框架, 这是首个面向漏洞检测的知识级 RAG 方法。与直接检索相似代码片段的代码级 RAG 不同, VUL-RAG 从历史 CVE 实例中蒸馏出包含功能语义 (purpose/function)、漏洞成因 (vulnerability cause) 与修复方案 (solution) 的多维结构化知识, 并设计“知识库构建—语义检索—知识增强推理”三阶段 workflow。在 PairVul 基准上, VUL-RAG 将 LLM 的 Pair Accuracy 提升了 16%–24%, 平衡精确率与召回率分别提升 9%–14% 与 7%–11% [7]。

尽管 VUL-RAG 在整体性能上表现优异, 本文在对照其开源实现进行复现的过程中发现, 检测决策层存在结构性缺陷。原始实现采用早退策略: 按检索排序依次评估每条知识, 一旦某条知识满足“存在漏洞成因且未应用修复措施” (`vul_result=1`, `sol_result=0`), 即立即判定为有漏洞; 若开启 `--early_return` 且检测到修复措施, 则立即判定为安全; 否则返回“无法确定” (`final_result=-1`)。这种“一票否决制”使得排名靠后、相似度较低的噪声知识同样拥有与首位知识对等的决策权。论文附录中的误报 (False Positive) 分析亦指出, 相当比例的误报源于不相关知识的检索或知识描述的不精确 [7]。在本地轻量化模型 (参数量 7B–16B) 复现场景下, 该问题更为突出, 严重制约了 Pair Accuracy 的提升空间。

针对上述问题, 本文提出 **倒数排名加权综合决策方法 WRAD** (Weighted Reciprocal-rank Aggregated Decision)。核心思想是: 不再由单一知识项“一票否决”, 而是对全部检索知识赋予与 BM25 排序一致的倒数排名权重, 分别累加漏洞证据与修复证据的加权强度, 通过全局比较做出更稳健的二分类判定。该方法仅修改 `detect_code` 函数的决策层, 保留所有 prompt 模板、LLM 调用与检索逻辑, 对上下游模块零侵入。

本文的主要贡献总结如下。

1. **系统分析了 VUL-RAG 原始决策层的“一票否决”问题**, 从算法逻辑与实验现象两个层面论证其对误报与 Pair Accuracy 的负面影响, 为决策层改进提供了明确的问题定义。
2. **提出 WRAD 加权综合决策方法**, 采用倒数排名权重 (Reciprocal Rank Weighting) 融合全部检索知识的漏洞/修复判定结果, 在无需训练与调参的前提下提升决策鲁棒性与结果确定性。

3. 完成 VUL-RAG 在 CWE-401 场景下的本地复现与改进验证, 并补充 CWE-264 跨类别实验, 基于 Ollama 部署的 DeepSeek-Coder-V2 与 Qwen2.5-Coder 模型, 系统对比不同知识来源、上下文长度与模型组合下的检测效果, 同时分析 WRAD 在另一类漏洞数据上的适用性与局限性.
4. 开源可复现的改进实现, 改进后的 `detect_code` 函数及完整流水线脚本可直接替换原 VUL-RAG 实现中的对应模块, 便于后续研究与工程落地.

本文第 2 节介绍 VUL-RAG 方法、CWE-401 检测场景及相关工作. 第 3 节详细阐述 WRAD 方法的设计与实现. 第 4 节给出实验设计、复现结果与对比分析. 第 5 节总结全文并展望未来工作.

2 研究背景

2.1 基于 LLM 的漏洞检测

LLM 在代码漏洞检测中的应用可分为 3 类典型范式.

预训练语言模型与深度学习检测. 研究者利用 Transformer 等预训练模型学习代码表示, 并在漏洞数据集上训练或评估漏洞分类模型 [8, 9]. 该方法能够学习比人工规则更丰富的代码特征, 但仍依赖训练数据分布, 对跨项目和跨漏洞类型泛化较为敏感.

微调 (Fine-tuning). 在漏洞数据集上对 LLM 进行监督微调 [10, 11]. 微调可提升特定数据集上的性能, 但泛化至新 CWE 类别或新代码库时效果下降, 且训练成本较高.

检索增强生成 (RAG). 从外部知识库检索相关信息辅助 LLM 推理 [12, 13]. 在漏洞检测领域, RAG 可分为代码级 RAG (检索相似代码片段) 与知识级 RAG (检索结构化漏洞知识). VUL-RAG [7] 属于后者, 通过高层次的漏洞行为描述引导 LLM 进行因果推理, 而非简单的文本相似度匹配. 相关研究还包括利用 RAG 扩充漏洞样本的 VulScribeR, 以及结合来源追踪开展漏洞分析的 ProveRAG [17, 18].

2.2 VUL-RAG 方法概述

VUL-RAG 包含 3 个核心阶段, 其总体流程如图 1 所示.

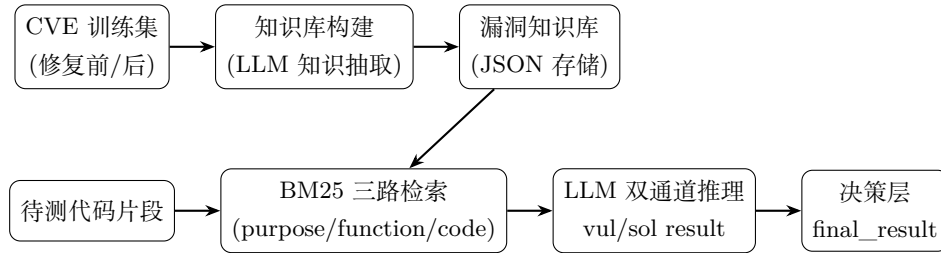


图 1 VUL-RAG 漏洞检测总体流程

阶段 1: 知识库构建. 对训练集中每个 CVE 的漏洞代码, 使用 LLM 抽取 purpose (函数目的)、function (功能列表)、vulnerability_behavior (漏洞成因、触发条件、具体代码行为) 与 solution (修复方案), 形成结构化 JSON 知识条目.

阶段 2: 知识检索. 对待测代码, 首先用 LLM 抽取其 purpose 与 function, 随后通过 BM25 算法分别在 purpose、function 与 code 三个维度检索知识库, 融合排序后选取 top-K 条 CVE 知识. 在每个 CVE 内部, 进一步对知识条目进行二级 BM25 排序, 选取最匹配的单条知识.

阶段 3: 知识增强推理与决策. 对每条检索到的知识, 分别构造两个 prompt: (a) 漏洞成因检测 prompt; (b) 修复措施检测 prompt. LLM 输出 `<result> YES </result>` 或 `<result> NO </result>`, 解析为二值结果 `vul_result` 与 `sol_result`. 决策层根据这些结果输出 `final_result`.

2.3 CWE-401 与内存泄漏检测

CWE-401 (Missing Release of Memory after Effective Lifetime) 描述因内存释放不当导致的内存泄漏问题, 在 Linux 内核等 C/C++ 项目中较为常见 [14]. 在 VUL-RAG 的 PairVul 基准中, CWE-401 包含 76 个 CVE、101 对漏洞/修复代码对 [7]. 内存泄漏漏洞的检测难点在于: 漏洞代码与修复代码在结构上高度相似, 差异往往仅体现在若干行 `kfree()`、`devm_kfree()` 等释放操作上, 纯语法或文本相似度难以有效区分.

VUL-RAG 论文报告, 在 DeepSeek-V2-Instruct 模型下, VUL-RAG 在全部 CWE 上的 Pair Accuracy 达 0.30, 平衡召回率与精确率分别为 0.61 与 0.62 [7]. 针对 CWE-401 单类, 论文 Figure 5(d) 显示 VUL-RAG 相比基线有显著提升.

2.4 原始决策层的问题

VUL-RAG 开源实现中 `detect_code` 函数的原始决策逻辑如算法 1 所示.

Algorithm 1 原始早退决策 (Original Early-Return Decision)

Require: 检索知识列表 $K = [k_1, k_2, \dots, k_n]$, 早退标志 `early_return`

Ensure: `final_result` $\in \{-1, 0, 1\}$

```

1: for  $i = 1$  to  $n$  do
2:    $(vul_i, sol_i) \leftarrow \text{LLM\_DETECT}(code, k_i)$ 
3:   if  $vul_i = 1$  and  $sol_i = 0$  then
4:     return 1 {立即判定有漏洞}
5:   else if  $sol_i = 1$  and early_return then
6:     return 0 {立即判定安全}
7:   end if
8: end for
9: return -1 {无法确定}

```

该策略存在 3 方面问题: (1) **低错误容忍度**, 任一知识项的 LLM 误判即可决定全局结果; (2) **信息利用不充分**, 早退机制导致通常仅评估前 1-2 条知识即返回; (3) **输出不确定**, 存在 `final_result` = -1 的三值输出.

表 1 从多个维度对比了原始方案与本文 WRAD 方案.

表 1 决策策略对比

对比维度	原方案 (早退 + 一票否决)	WRAD (加权综合评估)
决策依据	单一知识项, 首个匹配即决定	全部知识项的加权综合得分
排名影响	仅影响评估顺序	排名越靠前, 权重越大
错误容忍度	低	高: 单条错误被权重稀释
信息利用率	低	高: 评估全部知识
输出确定性	存在 -1	恒为 0 或 1
计算开销	较低	较高

3 WRAD 方法设计与实现

3.1 设计思想

WRAD 的核心思想是：让 **BM25 检索排序本身成为决策置信度的先验**，排名靠前的知识拥有更大话语权，但排名靠后的知识仍能以较小权重参与综合判断。最终通过比较“漏洞证据总强度”与“修复证据总强度”做出全局判定，而非被任意单条知识“一票否决”。

形式化地，设检索知识列表为 $K = [k_1, k_2, \dots, k_n]$ ，对第 i 条知识 (i 从 1 开始)，定义倒数排名权重：

$$W_i = \frac{1}{i} \quad (1)$$

对每条知识，LLM 双通道推理得到 $vul_result_i \in \{0, 1\}$ 与 $sol_result_i \in \{0, 1\}$ ，累加加权得分：

$$weighted_vul = \sum_{i=1}^n W_i \cdot vul_result_i \quad (2)$$

$$weighted_sol = \sum_{i=1}^n W_i \cdot sol_result_i \quad (3)$$

最终判定规则为：

$$final_result = \begin{cases} 1, & \text{if } weighted_vul > weighted_sol \\ 0, & \text{otherwise} \end{cases} \quad (4)$$

当两者相等时，判定为安全（保守策略，避免误报）。

3.2 算法描述

WRAD 加权综合决策如算法 2 所示。与算法 1 相比，算法 2 移除了循环体内的全部 **return** 语句，改为累加后继续迭代；循环结束后统一判定。

Algorithm 2 WRAD 加权综合决策

Require: 检索知识列表 $K = [k_1, k_2, \dots, k_n]$

Ensure: $final_result \in \{0, 1\}$, $weighted_vul$, $weighted_sol$

```

1:  $weighted\_vul \leftarrow 0, weighted\_sol \leftarrow 0, total\_weight \leftarrow 0$ 
2: for  $i = 1$  to  $n$  do
3:    $W_i \leftarrow 1/i$ 
4:    $(vul_i, sol_i) \leftarrow LLM\_DETECT(code, k_i)$ 
5:    $weighted\_vul \leftarrow weighted\_vul + W_i \times vul_i$ 
6:    $weighted\_sol \leftarrow weighted\_sol + W_i \times sol_i$ 
7:    $total\_weight \leftarrow total\_weight + W_i$ 
8: end for
9: if  $total\_weight > 0$  then
10:   $final\_result \leftarrow 1$  if  $weighted\_vul > weighted\_sol$  else  $0$ 
11: else
12:   $final\_result \leftarrow 0$ 
13: end if
14: return  $final\_result, weighted\_vul, weighted\_sol$ 
```

3.3 实现要点

本文在 VUL-RAG 开源代码的 `vulnerability_detect.py` 中, 对 `detect_code` 函数进行了上述改造. 关键实现片段如代码清单 1 所示:

```

1 weighted_vul = 0.0
2 weighted_sol = 0.0
3 total_weight = 0.0
4
5 for idx, vul_knowledge in enumerate(knowledge_list):
6     rank = idx + 1
7     weight = 1.0 / rank # reciprocal rank weighting
8     vul_result = extract_result_from_output(vul_output)
9     sol_result = extract_result_from_output(sol_output)
10    weighted_vul += weight * vul_result
11    weighted_sol += weight * sol_result
12    total_weight += weight
13
14 if total_weight > 0:
15     final_result = 1 if weighted_vul > weighted_sol else 0
16 else:
17     final_result = 0

```

Listing 1 WRAD 决策层核心实现

兼容性说明. WRAD 保留知识抽取、BM25 检索、检测 prompt 与评估脚本不变, 仅 `detect_code` 的决策逻辑被替换. 改进后 `--early_return` 参数不再被使用.

计算开销. WRAD 需对全部 `max_knowledge` (默认 3) 条知识完成双通道推理, LLM 调用次数从“1-6 次 (可变)”变为“固定 6 次”. 在本地 Ollama 部署场景下, 该开销增加可接受.

3.4 方法讨论

倒数排名思想已被用于信息检索的排序融合, 典型方法是 Reciprocal Rank Fusion (RRF) [15]. 标准 RRF 采用 $1/(k + r)$ 融合多个排序列表; 本文仅借鉴其“排名越高、贡献越大”的思想, 针对单个 BM25 排序列表使用更直接的 $1/i$ 权重. 该设计无需训练, 与检索排序语义一致且衰减平滑. 潜在改进方向包括: 引入软阈值 θ ; 归一化得分; 针对不同 CWE 类别调整权重衰减策略.

4 实验设计与结果分析

4.1 实验环境与数据集

硬件与软件环境. 实验在 Windows 10 平台上进行, 使用 Ollama 框架本地部署 LLM 服务. 推理框架为 VUL-RAG 开源实现 (Python 3.x), BM25 检索基于 `utils/bm25_retriever` 模块.

模型配置. 论文原版使用 Qwen2.5-Coder-32B-Instruct 与 DeepSeek-V2-Instruct [7, 16]. 受本地算力限制, 本文采用量化轻量模型, 如表 2 所示.

数据集. 主要对比实验使用 VUL-RAG PairVul 基准中的 **CWE-401** 测试集, 共 26 对 (52 个代码片段). 为考察方法在不同漏洞类别上的可迁移性, 进一步加入 **CWE-264** 补充测试集, 共 31 对 (62 个代码片段). 两类实验均使用原版知识库与 DeepSeek-Coder-V2 16k 配置; CWE-401 同时保留原

表 2 本地复现模型配置

用途	论文原版	本地复现 (Ollama)	上下文
知识抽取	Qwen2.5-Coder-32B / DeepSeek-V2	qwen2.5-coder:7b-instruct-q4_K_M / deepseek-coder-v2:16b-lite	4k/16k
漏洞检测	Qwen2.5-Coder-32B / DeepSeek-V2	deepseek-coder-v2:16b-lite-instruct-q4_K_M / qwen2.5-coder:7b	16k/32k

始早退决策作为基线, CWE-264 仅运行 WRAD, 因而用于适用性观察而非改进幅度比较. 默认参数: `retrieval_top_k=20, max_knowledge=3, thread_pool_size=5`.

4.2 评价指标

单片段指标. 对修复前代码: 检出漏洞为 TP, 未检出为 FN; 对修复后代码: 未检出漏洞为 TN, 误报为 FP. 计算平衡精确率、平衡召回率、F1 与 Accuracy.

配对指标 (Pair Metrics).

- **pair_right:** 修复前判为有漏洞且修复后判为无漏洞 (理想情况)
- **pair_1:** 修复前与修复后均判为有漏洞
- **pair_0:** 修复前与修复后均判为无漏洞
- **pair_wrong:** 修复前判安全、修复后判漏洞
- **Pair Accuracy:** $pair_right / pair_total$

4.3 基线复现实验

表 3 给出“原版知识 + DeepSeek (16k)”配置下, 原始早退决策策略的复现结果.

表 3 基线复现结果 (原版知识, DeepSeek 16k, 早退决策)

指标	值
TP / FP / FN / TN	13 / 7 / 13 / 19
Precision / Recall / F1	0.6219 / 0.6154 / 0.6186
Accuracy	0.6154
pair_right / pair_1 / pair_0 / pair_wrong	8 / 5 / 11 / 2
Pair Accuracy	0.3077
pair_1_rate / pair_0_rate / false_pair_rate	0.1923 / 0.4231 / 0.0769

与 VUL-RAG 论文中 DeepSeek-V2 在全部 CWE 上的 Pair Accuracy 0.30 [7] 相比, 本地复现在 CWE-401 单类上的 0.3077 与之接近. 但 pair_0_rate 高达 0.4231, 说明近半数样本对中修复前代码被漏报.

表 4 汇总了不同配置下的 Pair Accuracy.

4.4 WRAD 改进效果对比

在与基线相同配置下, 将决策层替换为 WRAD, 结果如表 5 所示.

WRAD 将 Pair Accuracy 从 0.3077 提升至 0.3462, 相对提升 12.5%. FN 从 13 降至 6, pair_0 从 11 降至 5, 漏报显著降低. FP 从 7 增至 12, pair_1 从 5 增至 11, 误报有所增加. 尽管 FP 增加, 但 TP 增

表 4 不同配置下的 Pair Accuracy 对比

知识抽取	漏洞检测	Pair Accuracy
原版知识	DeepSeek (16k)	0.3077
原版知识	DeepSeek (8k)	0.1923
原版知识	Qwen (16k)	0.1538
原版知识	Qwen (32k)	0.1538
原版知识	Gemma 4:4B (16k)	0.1923
DeepSeek (16k)	DeepSeek (16k)	0.2308
Qwen (4k)	DeepSeek (16k)	0.2692
DeepSeek (4k)	Qwen (16k)	0.0385
DeepSeek (16k)	Qwen (16k)	0.1154
Qwen (4k)	Qwen (16k)	0.1538

表 5 WRAD 与基线对比 (原版知识, DeepSeek 16k)

指标	基线 (早退)	WRAD (加权)	变化
TP	13	20	+7
FP	7	12	+5
FN	13	6	-7
TN	19	14	-5
Precision	0.6219	0.6625	+0.0406
Recall	0.6154	0.6538	+0.0384
F1	0.6186	0.6581	+0.0395
Accuracy	0.6154	0.6538	+0.0384
pair_right	8	9	+1
pair_1	5	11	+6
pair_0	11	5	-6
pair_wrong	2	1	-1
Pair Accuracy	0.3077	0.3462	+0.0385
false_pair_rate	0.0769	0.0385	-0.0384

幅更大, Precision、Recall、F1 与 Accuracy 均有所提升.

4.5 跨 CWE 补充实验: CWE-264

为初步考察 WRAD 在不同漏洞类别上的适用性, 本文在 CWE-264 测试集上补充实验. 该实验包含 31 对漏洞/修复代码 (62 个代码片段), 使用原版知识库、DeepSeek-Coder-V2 16k 上下文与 WRAD 决策, 结果如表 6 所示.

表 6 CWE-264 补充实验结果 (原版知识, DeepSeek 16k, WRAD)

指标	值
TP / FP / FN / TN	17 / 15 / 14 / 16
Precision / Recall / F1	0.5323 / 0.5323 / 0.5323
Accuracy	0.5323
pair_right / pair_1 / pair_0 / pair_wrong	10 / 7 / 6 / 8
Pair Accuracy	0.3226
pair_1_rate / pair_0_rate / false_pair_rate	0.2258 / 0.1935 / 0.2581

CWE-264 上的 Pair Accuracy 为 0.3226, 与 CWE-401 在 WRAD 下的 0.3462 相差 0.0236, 表明相同决策逻辑能够直接迁移到另一类漏洞数据并获得接近的配对识别水平. 但该结果不能单独证明 WRAD 在 CWE-264 上优于原始决策: 本次实验缺少同知识库、同模型与同推理参数下的早退基线. 此外, CWE-264 的 pair_wrong 为 8, false_pair_rate 达 0.2581, 明显高于 CWE-401 的 0.0385, 说明跨类别迁移后仍存在较强的方向性误判, 方法的稳定性和泛化性仍需更多 CWE 与基线实验验证.

4.6 结果讨论

WRAD 为何提升 Pair Accuracy. 原始早退策略中, 一条噪声知识即可触发 $vul=1$, $sol=0$ 并立即返回. WRAD 通过权重稀释, 要求多条高权重知识共同指向漏洞; 同时消除“先遇到 $sol=1$ 而早退判安全”的情况, 降低 pair_0.

pair_1 上升的原因. WRAD 判定条件为 $weighted_vul > weighted_sol$, 而非“存在漏洞且不存在修复”. 修复后代码中, 若多条知识的 vul_result 均为 1 而 sol_result 不全为 1, 仍可能判为有漏洞.

跨 CWE 结果的含义. CWE-264 的 Pair Accuracy 与 CWE-401 接近, 说明 WRAD 不依赖 CWE-401 特有的内存释放语义即可运行; 但较高的 false_pair_rate 表明不同 CWE 的证据分布和修复模式存在差异, 单一的 $1/i$ 权重与统一判定阈值未必适用于所有类别. 因此, 本文只能声称获得了跨 CWE 适用性的初步证据, 尚不能声称已验证普遍有效性.

与论文差距分析. VUL-RAG 论文使用 DeepSeek-V2-Instruct (671B MoE), 本文使用 16B 量化模型, 与论文 Figure 5(d) 中 CWE-401 结果相比仍存在差距, 主要源于模型规模、量化精度与推理参数差异.

未来改进方向. 可引入差值阈值 θ 、对 sol_result 赋予更高权重、结合 BM25 分数联合加权, 或在更大规模模型上验证 WRAD 的上限性能.

5 总结

本文针对 VUL-RAG 漏洞检测框架中决策层“一票否决”与早退策略导致的鲁棒性不足问题, 提出了倒数排名加权综合决策方法 WRAD. 该方法保留 VUL-RAG 全部知识抽取、检索与 LLM 推理流程, 仅替换 detect_code 函数的决策逻辑. 在 CWE-401 内存泄漏场景下的本地复现实验表明, WRAD 将

Pair Accuracy 从 0.3077 提升至 0.3462, 真阳性增加 7 例, 漏报减少 7 例, F1 提升 0.0395. 在 CWE-264 的 31 对样本上, 相同的 WRAD 配置获得 0.3226 的 Pair Accuracy, 提供了方法可迁移到另一漏洞类别的初步证据. 同时, 实验也揭示 WRAD 可能增加修复后代码的误报 (pair_1 上升), 且 CWE-264 的 false_pair_rate 达 0.2581. 由于 CWE-264 缺少同配置早退基线, 本文不将该结果解释为跨类别性能提升, 而仅作为适用性补充验证.

后续研究将补充 CWE-264 原始决策基线, 并在更多 CWE 类别上开展成对对照实验, 进一步探索软阈值、修复证据加权与 BM25 分数联合加权等增强策略, 以及在更大规模模型上复现并逼近 VUL-RAG 论文报告的性能上界.

参考文献

- [1] Li Z, Zou D, Xu S, et al. VulDeePecker: A deep learning-based system for vulnerability detection. In: Proc. of the 25th Annual Network and Distributed System Security Symp. (NDSS). San Diego: Internet Society, 2018. doi:10.14722/ndss.2018.23158.
- [2] Wu Y, Zou D, Dou S, et al. VulCNN: An image-inspired scalable vulnerability detection system. In: Proc. of the 44th IEEE/ACM Int'l Conf. on Software Engineering (ICSE). New York: ACM, 2022. 2365–2376. doi:10.1145/3510003.3510229.
- [3] Bessey A, Block K, Chelf B, et al. A few billion lines of code later: Using static analysis to find bugs in the real world. Communications of the ACM, 2010, 53(2): 66–75. doi:10.1145/1646353.1646374.
- [4] Chen Y, Ding Z, Alowain L, Chen X, Wagner D. DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability detection. In: Proc. of the 26th Int'l Symp. on Research in Attacks, Intrusions and Defenses (RAID). Hong Kong: ACM, 2023. 654–668. doi:10.1145/3607199.3607242.
- [5] OpenAI. GPT-4 technical report. arXiv preprint arXiv:2303.08774, 2023. <https://arxiv.org/abs/2303.08774>.
- [6] Hui B, Yang J, Cui Z, et al. Qwen2.5-Coder technical report. arXiv preprint arXiv:2409.12186, 2024. <https://arxiv.org/abs/2409.12186>.
- [7] Du X, Zheng G, Wang K, et al. Vul-RAG: Enhancing LLM-based vulnerability detection via knowledge-level RAG. ACM Transactions on Software Engineering and Methodology, 2026. doi:10.1145/3797277.
- [8] Thapa C, Jang S I, Ahmed M E, et al. Transformer-based language models for software vulnerability detection. In: Proc. of the 38th Annual Computer Security Applications Conf. (ACSAC). Austin: ACM, 2022. 481–496. doi:10.1145/3564625.3567985.
- [9] Steenhoek B, Rahman M M, Jiles R, Le W. An empirical study of deep learning models for vulnerability detection. In: Proc. of the 45th IEEE/ACM Int'l Conf. on Software Engineering (ICSE). Melbourne: IEEE, 2023. 2237–2248. doi:10.1109/ICSE48619.2023.00188.
- [10] Fu M, Tantithamthavorn C. LineVul: A transformer-based line-level vulnerability prediction. In: Proc. of the 19th Int'l Conf. on Mining Software Repositories (MSR). Pittsburgh: ACM, 2022. 608–620. doi:10.1145/3524842.3528452.
- [11] Shestov A, Levichev R, Mussabayev R, et al. Finetuning large language models for vulnerability detection. arXiv preprint arXiv:2401.17010, 2024. <https://arxiv.org/abs/2401.17010>.
- [12] Lewis P, Perez E, Piktus A, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems 33 (NeurIPS). Vancouver: Curran Associates, 2020. 9459–9474.

- [13] Gao Y, Xiong Y, Gao X, et al. Retrieval-augmented generation for large language models: A survey. arXiv preprint arXiv:2312.10997, 2023. <https://arxiv.org/abs/2312.10997>.
- [14] MITRE. CWE-401: Missing release of memory after effective lifetime (CWE version 4.20) [EB/OL]. 2026 [2026-06-28]. <https://cwe.mitre.org/data/definitions/401.html>.
- [15] Cormack G V, Clarke C L A, Buettcher S. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In: Proc. of the 32nd Annual Int'l ACM SIGIR Conf. on Research and Development in Information Retrieval. Boston: ACM, 2009. 758–759. doi:10.1145/1571941.1572114.
- [16] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. arXiv preprint arXiv:2405.04434, 2024. <https://arxiv.org/abs/2405.04434>.
- [17] Daneshvar S S, Nong Y, Yang X, et al. VulScribeR: Exploring RAG-based vulnerability augmentation with LLMs. ACM Transactions on Software Engineering and Methodology, 2026, 35(5): Article 125, 1–26. doi:10.1145/3760775.
- [18] Fayyazi R, Trueba S H, Zuzak M, Yang S J. ProveRAG: Provenance-driven vulnerability analysis with automated retrieval-augmented LLMs. IEEE Access, 2025, 13: 212815–212826. doi:10.1109/ACCESS.2025.3638251.

A 实验流水线命令

A.1 知识抽取

```
1 python extract_knowledge.py \  
2     --input_file_name CWE-401_train.json \  
3     --output_file_name CWE-401_knowledge.json \  
4     --model_name qwen2.5-coder:7b-instruct-q4_K_M \  
5     --model_settings "num_ctx=4096" \  
6     --thread_pool_size 5
```

A.2 漏洞检测

```
1 python vulnerability_detect.py \  
2     --input_file_name CWE-401_test.json \  
3     --knowledge_file_name CWE-401_knowledge.json \  
4     --output_file_name CWE-401_detect_result.json \  
5     --model_name deepseek-coder-v2:16b-lite-instruct-q4_K_M \  
6     --summary_model_name deepseek-coder-v2:16b-lite-instruct-q4_K_M \  
7     --model_settings "num_ctx=16384" \  
8     --max_knowledge 3 \  
9     --thread_pool_size 5
```

A.3 结果评估

```
1 python evaluate_result.py \  
2     --input_files CWE-401_detect_result.json
```